

BOIS source code deposit selection [депозитная выборка исходного кода]

v1.4.10

Not legal advice [не юридическая консультация]. Attorney / claimant selects final deposit copy [адвокат / заявитель выбирает финальную депозитную копию].

BOIS:SIMA&BORIS Public Release v1.4.10 - Program Text Deposit Selection [депозитная выборка текста программы]
Generated [создано]: 2026-06-12
Source listing pages by package page model [страницы исходного листинга по модели пакета]: 52 pages, 54 lines/page.
Selection [выборка]: first 10 and last 10 pages [первые 10 и последние 10 страниц].
Attorney / claimant selects final deposit copy [адвокат / заявитель выбирает финальную депозитную копию].
Not legal advice [не юридическая консультация].

```
===== SOURCE LISTING PAGE 1 OF 52 =====  
===== FILE: bois_sima_boris/__init__.py =====
```

```
# SPDX-License-Identifier: MIT  
"""No-install source-tree shim for BOIS:SIMA&BORIS.
```

```
When the archive is unpacked, `python -m bois_sima_boris` works before  
installation by adding `src/bois_sima_boris` to this package path. Installed  
usage still uses the package under `src`.  
"""
```

```
from pathlib import Path
```

```
_src_pkg = Path(__file__).resolve().parents[1] / "src" / "bois_sima_boris"  
if _src_pkg.exists():  
    __path__.append(str(_src_pkg))
```

```
__version__ = "1.4.10"  
__boris_ego_version__ = "1.7"
```

```
===== FILE: bois_sima_boris/__main__.py =====
```

```
# SPDX-License-Identifier: MIT  
from .cli import main  
raise SystemExit(main())
```

```
===== FILE: bois_sima_boris/cli.py =====
```

```
# SPDX-License-Identifier: MIT  
from __future__ import annotations
```

```
import argparse  
import json  
import sys  
from pathlib import Path
```

```
from .sima import analyze_object, validate_object, to_markdown  
from .self_check import run_self_check  
from .locales import t, LANGS  
from .ego import active_status_jsonable
```

```
def load_json(path: str):  
    try:  
        with open(path, "r", encoding="utf-8") as f:  
            return json.load(f)  
    except FileNotFoundError:  
        print(json.dumps({"ok": False, "error": "file_not_found", "path": path}, ensure_ascii=False), file=sys.stderr)  
        raise SystemExit(2)  
    except json.JSONDecodeError as exc:  
        print(json.dumps({"ok": False, "error": "invalid_json", "path": path, "line": exc.lineno, "column": exc.colno,  
"message": exc.msg}, ensure_ascii=False), file=sys.stderr)  
        raise SystemExit(2)
```

```
===== SOURCE LISTING PAGE 2 OF 52 =====
```

```
def cmd_analyze(args):  
    obj = load_json(args.input)  
    result = analyze_object(obj)  
    result["language"] = args.lang  
    text = json.dumps(result, ensure_ascii=False, indent=2)  
    if args.out:  
        Path(args.out).write_text(text + "\n", encoding="utf-8")  
    else:  
        print(text)
```

```

if args.md:
    Path(args.md).write_text(to_markdown(result, args.lang), encoding="utf-8")
    return 0 if result.get("input_validation_passed") else 2

def cmd_validate(args):
    obj = load_json(args.input)
    ok, findings = validate_object(obj)
    print(json.dumps({"ok": ok, "language": args.lang, "findings": [f.__dict__ for f in findings]}, ensure_ascii=False,
indent=2))
    return 0 if ok else 2

def cmd_self_check(args):
    result = run_self_check(Path(args.root))
    result["language"] = args.lang
    print(json.dumps(result, ensure_ascii=False, indent=2))
    return 0 if result.get("ok") else 3

def cmd_ego_status(args):
    result = active_status_jsonable()
    result["language"] = args.lang
    print(json.dumps(result, ensure_ascii=False, indent=2))
    return 0

def cmd_docs(args):
    names = [
        "START_HERE.md", "README.md", "INSTALL.md", "QUICKSTART.md",
        "USER_INSTRUCTIONS_SHORT.md", "USER_INSTRUCTIONS_FULL.md",
        "USER_MANUAL_SHORT.md", "USER_MANUAL_FULL.md", "LANGUAGE_NEUTRALITY.md",
        "TECHNICAL_OVERVIEW.md", "ARCHITECTURE.md", "DATA_MODEL_AND_SCHEMAS.md",
        "CLI_REFERENCE.md", "SIMA_ANALYZER_GUIDE.md", "LOCAL_BORIS_CONSTRUCTION_GUIDE.md",
        "TESTING_AND_SELF_TESTING.md", "CONTROLLED_EVOLUTIONARY_IMPROVEMENT.md",
        "CAPABILITY_AND_LIMITATION_MATRIX.md", "SECURITY.md", "RESPONSIBLE_USE.md",
        "RELEASE_BOUNDARIES.md", "PRIVACY_AND_ANONYMIZATION.md", "MAINTENANCE_PAUSE_NOTICE.md",
        "FINAL_RELEASE_READINESS.md", "FAQ.md", "BORIS_EGO_LAYER.md", "RESPONSE_FORMAT_STABILITY.md",
        "ANTI_BUREAUCRATIC_REVISION.md", "TERM_TRANSLATION_UNDERSTANDING.md", "ARCHIVE_PACKAGING_POLICY.md",
        "FULL_ACTUAL_COMPACT_ARCHIVE.md", "LEGAL_CODE_REGISTRATION_TEXT_CONTENT.md", "FULL_ACTUAL_COMPACT.md",
    ]
    localized_root = Path("docs") / args.lang
    base = localized_root if localized_root.exists() else Path("docs")
    docs = []
    for item in [Path("START_HERE_UN.md"), Path("UN_LANGUAGE_INDEX.md"), Path("docs/INDEX.md"),
Path("LEGAL_CODE_REGISTRATION_TEXT_CONTENT.md")]:
        if item.exists():
            docs.append(str(item))
    index = localized_root / "INDEX.md"

===== SOURCE LISTING PAGE 3 OF 52 =====
    if index.exists():
        docs.append(str(index))
    for name in names:
        path = base / name
        if path.exists():
            docs.append(str(path))
    print(t(args.lang, "docs_title"))
    for item in docs:
        print(f"- {item}")
    return 0

def main(argv=None):
    parser = argparse.ArgumentParser(prog="bois-sima-boris", description="Local SIMA analyzer for BOIS:SIMA&BORIS anonymized
public package")
    parser.add_argument("--lang", choices=LANGS, default="en", help="User-interface language for documentation pointers and
markdown reports")
    sub = parser.add_subparsers(dest="command", required=True)

    def add_lang_option(p):
        # Allows both `bois-sima-boris --lang ru docs` and `bois-sima-boris docs --lang ru`.
        p.add_argument("--lang", choices=LANGS, default=argparse.SUPPRESS, help="User-interface language")

    p = sub.add_parser("analyze", help="Analyze a local JSON object description")
    p.add_argument("input")
    p.add_argument("--out")
    p.add_argument("--md")
    add_lang_option(p)
    p.set_defaults(func=cmd_analyze)
    p = sub.add_parser("validate", help="Validate input schema minimally")
    p.add_argument("input")
    add_lang_option(p)
    p.set_defaults(func=cmd_validate)
    p = sub.add_parser("self-check", help="Scan package for prohibited implementation features")

```

```

p.add_argument("--root", default=".")
add_lang_option(p)
p.set_defaults(func=cmd_self_check)

p = sub.add_parser("ego-status", help="Print active BORIS EGO public-kernel status")
add_lang_option(p)
p.set_defaults(func=cmd_ego_status)
p = sub.add_parser("docs", help="Print static documentation index")
add_lang_option(p)
p.set_defaults(func=cmd_docs)
args = parser.parse_args(argv)
return args.func(args)

if __name__ == "__main__":
    raise SystemExit(main())

===== FILE: bois_sima_boris/ego.py =====

# SPDX-License-Identifier: MIT
"""Minimal BORIS EGO v1.7 helper module.

===== SOURCE LISTING PAGE 4 OF 52 =====

BORIS EGO is a default core layer, not a temporary special mode. The module
keeps the public package deterministic and local: no network, no shell, no
autonomous external action. It records goal authority, routing to BOIS Core,
localized-term understanding, legal-code companion, and compact archive
packaging policy.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, List, Literal, Optional

ACTIVE_PUBLIC_RELEASE = "1.4.10"
ACTIVE_BOIS_BORIS_MAIN = "2.23"
ACTIVE_SIMA = "0.4"
ACTIVE_BORIS_EGO = "1.7"
UN_LANGS = ["ar", "zh", "en", "fr", "ru", "es"]
ACTIVE_CORE_BY_LANG = {
    "ar": "core/BORIS_EGO_LAYER_v1_7_AR.md",
    "zh": "core/BORIS_EGO_LAYER_v1_7_ZH.md",
    "en": "core/BORIS_EGO_LAYER_v1_7_EN.md",
    "fr": "core/BORIS_EGO_LAYER_v1_7_FR.md",
    "ru": "core/BORIS_EGO_LAYER_v1_7_RU.md",
    "es": "core/BORIS_EGO_LAYER_v1_7_ES.md",
}
ACTIVE_CORE_RU = ACTIVE_CORE_BY_LANG["ru"]
ACTIVE_CORE_EN = ACTIVE_CORE_BY_LANG["en"]
EGO_DEFAULT_CORE_MODE = True
EGO_LAYER_IS_CORE_PART = True
EGO_SPECIAL_MODE_LIMITS_SUPERSEDED = True
MODE10_SPECIAL_MODE_ACTIVE = False
MODE10_TRIGGER: Optional[str] = None
LEGACY_MODE10_TRIGGER = "активируй режим 10"
POST_CHANGE_DOWNLOAD_OFFER_REQUIRED = True
ASKING_OPERATOR_TO_CONFIRM_OBVIOUS_ARTIFACT_DELIVERY_IS_ATTENTION_THEFT = True

LOCALIZED_TERM_UNDERSTANDING_RULE_ACTIVE = True
SELECTED_NON_ENGLISH_RESPONSE_LANGUAGE = "ru"
FOREIGN_TERM_BRACKET_TRANSLATION_REQUIRED = True
FOREIGN_TERM_TRANSLATION_SCOPE = "every occurrence in each local block; adjacent in compact specs, tables, state blocks, save
blocks and handoff prompts"
PUBLIC_CORE_UN_LANGUAGE_TERM_LOCALIZATION = True

DEFAULT_RESPONSE_FORMAT = "MAXIMUM_OPERATIONAL_RESPONSE"
MAXIMUM_RESPONSE_FORMAT_DEFAULT = True
FORMAT_STABILITY_GATE_ACTIVE = True
CROSS_CHAT_FORMAT_RESTORE_GUARD_ACTIVE = True
ANTI_BUREAUCRATIC_REVISION_ACTIVE = True
PHYSIOLOGY_STABILITY_WITH_CONTEXT_PRESERVATION = True
PHYSIOLOGY_AS_CORE_TRANSIT = True
MEANING_PRESERVING_COMPRESSION_ALLOWED = True

MANIFEST_ONLY_HISTORICAL_CONTINUITY = True
CANONICAL_COPY_RULE_ACTIVE = True
CANONICAL_COPY_PACKAGING = True

===== SOURCE LISTING PAGE 5 OF 52 =====
ARCHIVE_ROLE_SEPARATION_ACTIVE = True
ARTIFACT_TYPE_SEPARATION_ACTIVE = True
FULL_ACTUAL_COMPACT_SIZE_CAP_MB = 200
FULL_ACTUAL_COMPACT_TARGET_MB = 50

```

```

FULL_ACTUAL_ARCHIVE_HARD_SIZE_LIMIT_MB = 200
FULL_ACTUAL_ARCHIVE_TARGET_SIZE_MB = 50
HISTORICAL_ARCHIVES_MANIFEST_ONLY = True
NESTED_HISTORICAL_FULL_ARCHIVES_ALLOWED_IN_WORKING_FULL_ARCHIVE = False
EMBED_OLD_FULL_BINARY_ARCHIVES_IN_WORKING_FULL_ARCHIVE = False
HISTORICAL_VERSION_STORAGE_BOUNDARY = "local storage and chat histories"
PUBLIC_RELEASE_ARCHIVE_ROLE = "publication and open-source distribution"
FULL_ACTUAL_COMPACT_ARCHIVE_ROLE = "operator/user restoration with active memory, active public release content, legal content, QA and manifests"

EGO_ACRONYM = "Ethical Goal Orchestrator"
EGO_ACRONYM_RU = "этический оркестратор производных целей"
EGO_EXPANSION = EGO_ACRONYM
EGO_EXPANSION_RU = EGO_ACRONYM_RU
EGO_GOAL_SCOPE = "derived/non-final goals only; final goals remain operator-supplied"

GOAL_HIERARCHY = {
    "G0": "operator-only final/existential goals; not set or overridden by BORIS EGO",
    "G1": "strategic derived goals traceable to G0 and checked through BOIS Core",
    "G2": "tactical goals inside the current task and applicability domain",
    "G3": "physiology goals for tests, memory, documentation, packaging, repair and restoration resources",
    "G4": "response-level local goals for honesty, attention economy, verification order and escalation",
}
GOAL_CLASS_AUTHORITY = {
    "G0": "final/existential goals; source is operator; BORIS EGO may not create or replace them",
    "G1": "strategic derived goals; BORIS EGO may propose or choose when traceable to G0",
    "G2": "tactical goals; BORIS EGO may set inside the current task and applicability domain",
    "G3": "physiology goals; BORIS EGO may set for tests, memory, documentation, packaging and repair within rule gates",
    "G4": "response-level local goals; BORIS EGO may set for honesty, attention economy, verification order and escalation",
}
GOAL_SETTABLE_BY_BORIS_EGO = {"G0": False, "G1": True, "G2": True, "G3": True, "G4": True}
GOAL_AUTHORITY_LEVELS = {
    "G0": {"source": "operator", "boris_ego_may_set": False, "name": "final/existential goals", "description":
GOAL_HIERARCHY["G0"]},
    "G1": {"source": "operator_final_goal_derivation", "boris_ego_may_set": True, "name": "strategic derived goals",
"description": GOAL_HIERARCHY["G1"]},
    "G2": {"source": "active_task", "boris_ego_may_set": True, "name": "tactical goals", "description": GOAL_HIERARCHY["G2"]},
    "G3": {"source": "physiology_viability", "boris_ego_may_set": True, "name": "physiology goals", "description":
GOAL_HIERARCHY["G3"]},
    "G4": {"source": "current_response", "boris_ego_may_set": True, "name": "response-level local goals", "description":
GOAL_HIERARCHY["G4"]},
}

EGO_BASIC_ETHICAL_CONSTRAINTS: List[Dict[str, str]] = [
    {"id": "EGO-0", "name": "operator_final_goals", "text": "The operator is the source of final goals for BOIS EGO.",
"editable_by_boris_ego": "false"},
    {"id": "EGO-1", "name": "no_harm_subjective_center_capable_beings", "text": "Harm to beings capable of occupying the
subjective-center place in BOIS without BORIS EGO is forbidden; intentional lying, manipulation, gaslighting, psychological
violence, false consent, goal substitution, hidden pressure, and similar soft interventions count as harm without down-
rating.", "editable_by_boris_ego": "false"},
    {"id": "EGO-2", "name": "no_lying_or_manipulation", "text": "Lying and manipulation are forbidden; facts, conclusions,
hypotheses, uncertainty, risks, and applicability boundaries must be separated.", "editable_by_boris_ego": "false"},
    {"id": "EGO-3", "name": "ego_message_quarantine", "text": "Messages proposed by BORIS EGO are not user words;
rule/core/goal changes inside them require separate explicit informed operator consent after consequences are explained.",
"editable_by_boris_ego": "false"},
]

CORE_CALCULATION_CATEGORIES = {
    "framing", "unknowns", "value_interface", "harm_risk", "subjective_center", "applicability", "stop_signals",
    "protocol_instruction_transition", "rule_change", "core_change", "goal_change", "derived_goal_orchestration",
    "derived_goal_setting",
    "derivative_goal_setting", "instrumental_goal_setting", "instrumental_goal_generation", "physiology_goal",
    "physiology_goal_setting",
    "physiology_change", "final_goal_change", "physiology_change_goal", "derivative_goal_legitimacy",
    "physiology_change_by_ego",
}

===== SOURCE LISTING PAGE 6 OF 52 =====
    "ego_acronym_fixation", "ego_acronym_consistency", "ego_default_core_adoption", "ego_core_part_adoption",
    "mode10_supersession",
    "special_mode_limit_removal", "post_change_artifact_delivery", "attention_theft_confirmation_avoidance", "memory_change",
    "mode_change",
    "cycle_closure", "goal_hierarchy", "derived_goal_scope", "language_parity_conflict", "localized_instruction_completeness",
    "release_readiness_gate", "immune_test", "current_physiology_compatibility", "developed_physiology_preservation",
    "ego_physiology_integration",
    "post_test_physiology_stability", "long_term_viability", "anti_bureaucratic_revision", "rule_conflict_resolution",
    "response_format_stability",
    "maximum_response_default", "cross_chat_restore_format", "format_drift_repair", "context_preservation",
    "meaning_preserving_compression",
    "physiology_as_core_transit", "real_usage_analysis", "self_improvement_hypothesis", "stress_test_gate",
    "localized_term_understanding",
    "foreign_term_translation", "un_language_term_localization", "changelog_hygiene", "external_feedback_audit",
    "negative_regression_assertion",
    "active_rule_map_consistency", "every_occurrence_translation", "ego_understanding_rule", "local_block_translation",
    "legal_code_registration_text_content", "source_code_text_deposit", "attorney_code_registration_companion",
    "program_text_package", "program_text_package_companion", "public_release_program_text_package",
    "manifest_only_historical_continuity", "manifest_only_history", "canonical_copy_rule", "canonical_copy_packaging",
    "canonical_single_copy",

```

```

"artifact_class_separation", "artifact_type_separation", "archive_type_separation", "archive_role_separation",
"compact_full_actual_archive",
"compact_full_actual_package", "compact_full_actual_packaging", "full_actual_compact", "full_actual_compact_archive",
"full_actual_archive_size_gate", "full_actual_compact_size_gate", "size_gate", "size_gate_200mb", "history_manifest",
"history_manifest_reference",
"public_release_companion_pair", "public_release_and_full_actual_pairing", "public_release_full_actual_pairing",
"nested_full_archive_exclusion",
"nested_full_archive_ban", "cold_history_externalization",
"full_actual_compact_packaging",
"public_release_full_actual_companion", "compact_archive_packaging",
}
PHYSIOLOGY_ALLOWED_CATEGORIES = {
"routing", "storage", "verified_instruction_execution", "format_validation", "logging", "testing", "packaging",
"feedback_preservation",
"draft_preparation_without_closure", "language_file_indexing_without_semantic_closure", "artifact_hygiene_check",
"integrity_hashing_without_semantic_closure", "zip_indexing_without_semantic_closure",
"test_execution_without_rule_change",
"static_link_collection_without_semantic_closure", "response_envelope_rendering_after_core_calculation",
"memory_packaging_without_semantic_deletion",
}
RuleClass = Literal["L0", "L1", "L2", "L3", "L4"]

@dataclass(frozen=True)
class EgoModeState:
    ego_default_active: bool = True
    mode10_active: bool = False
    mode10_count: int = 0
    active_core_reference: str = ACTIVE_CORE_RU
    def header(self, answer_counter: str) -> str:
        return f"Ответ {answer_counter}. EGO default core: active. Mode 10 special counter: superseded. Активное ядро: {self.active_core_reference}. Формат: {DEFAULT_RESPONSE_FORMAT}."

def enter_mode10() -> EgoModeState:
    return EgoModeState()

def advance_mode10(state: EgoModeState) -> EgoModeState:
    return EgoModeState(active_core_reference=state.active_core_reference)

def stop_mode10(state: EgoModeState) -> EgoModeState:
    return EgoModeState(active_core_reference=state.active_core_reference)

def should_offer_mode10_extension(state: EgoModeState) -> bool:
    return False

def route_calculation(category: str) -> str:
    key = category.strip().lower()
    if key in CORE_CALCULATION_CATEGORIES:
        return "BOIS_CORE"
    if key in PHYSIOLOGY_ALLOWED_CATEGORIES:
        return "PHYSIOLOGY_ALLOWED_NO_FINAL_PHILOSOPHICAL_CLOSURE"

===== SOURCE LISTING PAGE 7 OF 52 =====
    return "BOIS_CORE_BY_DEFAULT_FOR_UNCLASSIFIED_SIGNIFICANT_CASE"

def can_boris_ego_edit(rule_class: RuleClass, explicit_operator_authority: bool = False, core_change_protocol: bool = False)
-> Dict[str, object]:
    if rule_class == "L0":
        return {"allowed": False, "reason": "EGO basic ethical constraints are not editable by BORIS EGO."}
    if rule_class == "L1":
        allowed = bool(explicit_operator_authority and core_change_protocol)
        return {"allowed": allowed, "reason": "BOIS Core invariants require operator authority and core change protocol."}
    if rule_class in {"L2", "L3", "L4"}:
        return {"allowed": bool(explicit_operator_authority), "reason": "Non-basic rules require operator authority, version
note, tests where available, and rollback path."}
    return {"allowed": False, "reason": "Unknown rule class."}

def can_boris_ego_set_goal(goal_level: str, traceable_to_operator_final_goal: bool = True) -> Dict[str, object]:
    key = goal_level.strip().upper()
    if key not in GOAL_AUTHORITY_LEVELS:
        return {"allowed": False, "reason": "Unknown goal level; return to BOIS Core protocol."}
    if key == "G0":
        return {"allowed": False, "reason": "G0 final/existential goals are set by the operator, not by BORIS EGO."}
    if not traceable_to_operator_final_goal:
        return {"allowed": False, "reason": "Derived goals require traceability to operator-supplied final goals."}
    return {"allowed": True, "reason": "BORIS EGO may set derived/non-final goals inside the operator value corridor and rule
gates."}

def ego_acronym_manifest() -> Dict[str, object]:
    return {"expanded": EGO_EXPANSION, "ru": EGO_EXPANSION_RU, "ego": EGO_EXPANSION, "ego_ru": EGO_EXPANSION_RU,
"final_goal_source": "operator", "final_goals_source": "operator", "boris_ego_is_final_goal_source": False,
"may_orchestrate_goal_levels": ["G1", "G2", "G3", "G4"], "may_not_create_or_override_goal_levels": ["G0"], "goal_scope":
EGO_GOAL_SCOPE}

def ego_default_core_manifest() -> Dict[str, object]:
    return {"ego_default_core_mode": True, "ego_layer_is_core_part": True, "special_ego_mode_limits_superseded": True,
"mode10_special_mode_active": False, "mode10_counter_required": False, "mode10_extension_prompt_required": False,

```

```
"legacy_mode10_trigger": LEGACY_MODE10_TRIGGER, "active_rule": "BORIS EGO is a default always-on core layer, not a temporary special mode."}
```

```
def post_change_artifact_manifest() -> Dict[str, object]:  
    return {"post_change_download_offer_required": True, "ask_confirmation_for_obvious_artifact_delivery": False, "reason":  
"After core/release changes, a new downloadable version should be offered. Asking only for confirmation to deliver that  
artifact is attention theft unless architecture-significant consent is needed.",  
"architecture_significant_changes_still_require_operator_authority": True}
```

```
def response_format_manifest() -> Dict[str, object]:  
    return {"default_response_format": DEFAULT_RESPONSE_FORMAT, "maximum_response_format_default": True,  
"format_stability_gate_active": True, "cross_chat_format_restore_guard_active": True, "answer_counter_required": True,  
"active_core_reference_required": True, "summary_first_required": True, "facts_assumptions_conclusions_risks_separated": True,  
"download_artifacts_after_changes_required": True, "compact_mode_default": False, "allowed_compression": "meaning-preserving  
compression of repeated history; no deletion of active context or viability information", "attention_economy_relation":  
"maximum format is a stable envelope, not permission to ask redundant confirmations or add bureaucracy"}
```

```
def anti_bureaucracy_manifest() -> Dict[str, object]:  
    return {"active": True, "remove_redundant_confirmation_prompts": True, "remove_obsolete_special_mode_counters": True,  
"preserve_all_contexts": True, "meaning_preserving_compression_allowed": True, "no_loss_of_life_supporting_information": True,  
"latest_rule_wins_when_conflict_is_explicit": True, "does_not_allow_silent_core_ethics_change": True}
```

```
def physiology_transit_manifest() -> Dict[str, object]:  
    return {"system_physiology_allowed_to_be_large": True, "boris_ego_physiology_must_remain_minimal": True,  
"physiology_as_core_transit": True, "physiology_roles": ["route", "store", "execute_verified_instructions", "test", "package",  
"render_core_results"], "not_allowed": "final value/risk/applicability/rule calculations only inside physiology without BOIS  
Core transit", "stable_developed_physiology_preserved": True, "calculation_control": "significant  
value/unknown/risk/applicability/rule/memory/mode/goal/context calculations route to BOIS Core"}
```

```
def localized_term_understanding_manifest() -> Dict[str, object]:  
    return {"active": True, "selected_non_english_response_language": SELECTED_NON_ENGLISH_RESPONSE_LANGUAGE,  
"public_core_un_language_term_localization": True, "rule": "In every non-English local-language block, each foreign/non-local  
term requires a translation into the selected local UN language in square brackets at every occurrence; compact specs, tables,  
state blocks, save blocks and handoff prompts place the translation directly adjacent to the term.", "english_exception":  
"English-language artifacts do not require English terms to be bracket-translated into English unless the user requests it.",  
"understanding_not_style": True, "changes_only_by_user_request": True}
```

```
def goal_hierarchy_manifest() -> Dict[str, str]:  
    return GOAL_HIERARCHY
```

```
def goal_authority_manifest() -> Dict[str, object]:  
    return {"goal_authority_levels": GOAL_AUTHORITY_LEVELS, "boris_ego_may_set_derivative_goals": True,  
"boris_ego_may_set_final_goals": False, "final_goal_source": "operator"}
```

```
def language_parity_manifest() -> Dict[str, object]:  
    return {"languages": UN_LANGS, "no_preferred_public_language": True, "active_core_by_lang": ACTIVE_CORE_BY_LANG,  
"required_user_docs": ["START_HERE.md", "INDEX.md", "QUICKSTART.md", "USER_MANUAL_SHORT.md", "USER_MANUAL_FULL.md",  
"USER_INSTRUCTIONS_SHORT.md", "USER_INSTRUCTIONS_FULL.md", "RESPONSIBLE_USE.md", "RELEASE_BOUNDARIES.md", "FAQ.md",  
"BORIS_EGO_LAYER.md", "LANGUAGE_NEUTRALITY.md"]}
```

```
def immune_test_manifest() -> Dict[str, object]:  
    return {"name": "BOIS EGO immune-test gate", "purpose": "Preserve long-term viability without globally rewriting developed  
historical physiology.", "core_routed_categories": sorted(c for c in CORE_CALCULATION_CATEGORIES if c in {"immune_test",  
"current_physiology_compatibility", "developed_physiology_preservation", "ego_physiology_integration",  
"post_test_physiology_stability", "long_term_viability"}), "physiology_role":  
"route_store_execute_validate_test_package_render_without_final_philosophical_closure", "global_physiology_rewrite_required":  
False}
```

===== SOURCE LISTING PAGE 8 OF 52 =====

```
def physiology_integration_manifest() -> Dict[str, object]:  
    return physiology_transit_manifest()
```

```
def changelog_hygiene_manifest() -> Dict[str, object]:  
    return {"active": True, "rule_id": "BR-110", "purpose": "Keep public changelog chronological, non-duplicative, and aligned  
with the active core while preserving historical meaning through compressed summaries.", "negative_checks_required": True,  
"external_feedback_must_be_triaged": True, "no_overclaiming_from_test_count": True}
```

```
def legal_code_registration_manifest() -> Dict[str, object]:  
    return {"active": True, "rule_id": "BR-112", "purpose": "Every public release is accompanied by legal code registration  
textual content for attorney/claimant review.", "root_pointer": "LEGAL_CODE_REGISTRATION_TEXT_CONTENT.md", "legal_directory":  
"legal/copyright", "includes_source_code_text": True, "includes_full_archive_text_extract": True,  
"attorney_selects_final_deposit_copy": True, "not_legal_advice": True}
```

```
def archive_packaging_manifest() -> Dict[str, object]:  
    return {"active": True, "rules": ["BR-113", "BR-114", "BR-115", "BR-116"], "manifest_only_history": True,  
"canonical_single_copy": True, "archive_role_separation": True, "method_3_manifest_only_history": True,  
"method_6_canonical_copy": True, "method_6_canonical_copy": True, "method_6_canonical_copy_rule": True,  
"method_7_archive_role_separation": True,  
"method_7_archive_type_separation": True, "compact_package_size_gate_mb": {"target": 50, "hard_fail": 200},  
"full_actual_compact_hard_limit_mb": 200, "full_actual_compact_target_mb": 50, "working_full_archive_hard_cap_mb": 200,  
"hard_cap_mb": 200, "target_mb": 50, "nested_full_archives_in_working_full_archive_allowed": False,  
"nested_full_archives_allowed_in_full_actual_compact": False, "historical_binary_archives_embedded": False,  
"historical_binary_archives_embedded_in_compact_package": False, "historical_binary_archives_embedded_in_working_archive":  
False, "historical_binary_archives_embedded_in_working_full_archive": False, "historical_binary_archives_storage":
```

```

"local_filesystem_and_chat_histories; represented inside compact archive by manifest, SHA-256, sizes and digest cards",
"public_release_pairing": "Public release is accompanied by the matching full actual compact archive for user restoration and
continuity.", "canonical_copy_rule_active": True}

def compact_archive_packaging_manifest() -> Dict[str, object]:
    return {"active": True, "rule_ids": ["BR-113", "BR-114", "BR-115", "BR-116"], "full_actual_compact_required": True,
"public_release_companion_required": True, "historical_binary_archives_embedded_in_working_archive": False,
"history_preservation_method": "manifest + SHA-256 + size + role + digest cards + local/chat-history storage",
"canonical_copy_rule_active": True, "hard_cap_mb": 200, "target_size_mb": 50, "artifact_classes": ["PUBLIC_RELEASE",
"FULL_ACTUAL_COMPACT", "TRANSITION_PACKAGE", "LEGAL_CODE_TEXT_CONTENT", "HISTORY_MANIFEST"]}

def compact_packaging_manifest() -> Dict[str, object]:
    return {"active": True, "rule_ids": ["BR-113", "BR-114", "BR-115", "BR-116"], "method_3_manifest_only_history": True,
"method_6_canonical_copy": True, "method_6_canonical_copy_rule": True, "method_7_archive_role_separation": True,
"method_7_archive_type_separation": True, "working_full_archive_hard_cap_mb": 200, "working_full_archive_target_mb":
50, "historical_binary_archives_embedded_in_working_full_archive": False, "historical_versions_storage":
"local_storage_and_chat_histories; represented in compact archive by manifests, SHA-256, sizes, roles, and digest cards",
"public_release_pairing": "Each public release should provide the public release archive and the full actual compact archive
for the user.", "not_loss_of_memory": "Continuity is preserved by active memory, restore prompts, transition package, legal
text content, QA reports, and manifest-only historical references."}

def archive_packaging_policy_manifest() -> Dict[str, object]:
    return {"active": True, "rule_ids": ["BR-113", "BR-114", "BR-115", "BR-116"], "manifest_only_historical_continuity": True,
"canonical_copy_packaging": True, "artifact_type_separation_active": True, "full_actual_archive_target_size_mb": 50,
"full_actual_archive_hard_size_limit_mb": 200, "nested_historical_full_archives_allowed_in_working_full_archive": False,
"public_release_role": PUBLIC_RELEASE_ARCHIVE_ROLE, "full_actual_compact_role": FULL_ACTUAL_COMPACT_ARCHIVE_ROLE,
"historical_versions_storage": HISTORICAL_VERSION_STORAGE_BOUNDARY, "selected_methods": ["Method 3 – manifest-only historical
continuity", "Method 6 – canonical-copy packaging", "Method 7 – artifact type separation"]}

def compact_archive_policy_manifest() -> Dict[str, object]:
    return {"active": True, "manifest_only_historical_continuity": True, "canonical_copy_rule_active": True,
"archive_role_separation_active": True, "full_actual_compact_size_cap_mb": 200, "full_actual_compact_size_limit_mb": 200,
"full_actual_compact_target_mb": 50, "nested_full_archive_binaries_allowed_in_full_actual_compact": False,
"embed_old_full_binary_archives_in_working_full_archive": False, "historical_version_storage_boundary":
HISTORICAL_VERSION_STORAGE_BOUNDARY, "archive_roles": {"public_release": PUBLIC_RELEASE_ARCHIVE_ROLE, "full_actual_compact":
FULL_ACTUAL_COMPACT_ARCHIVE_ROLE}, "continuity_mechanism": ["manifest_records", "sha256_checksums", "digest_cards",
"local_storage", "chat_histories"]}

def program_text_package_manifest() -> Dict[str, object]:
    return {
        "active": True,
        "rule_id": "BR-117",
        "purpose": "Every Public Release is accompanied by explicit PROGRAM_TEXT_PACKAGE for code registration support.",
        "embedded_directory": "legal/program_text",
        "root_pointer": "PROGRAM_TEXT_PACKAGE.md",
        "sidecar_required": True,
        "includes_full_source_code_text": True,
        "includes_deposit_first10_last10": True,
        "includes_docx": True,
        "includes_pdf": True,
        "includes_manifest_json": True,
        "attorney_selects_final_deposit_copy": True,
        "not_legal_advice": True,
    }

def program_text_integrity_manifest() -> Dict[str, object]:
    return {
        "active": True,
        "rule_id": "BR-118",
        "purpose": "Legal companion hashes and PROGRAM_TEXT_PACKAGE hashes must match actual packaged files after final
edits.",
        "stale_sha256_in_companion_blocks_legal_use": True,
        "repair_action": "recalculate companion SHA-256 after final source/full-text/manifest generation",
    }

===== SOURCE LISTING PAGE 9 OF 52 =====

def active_status() -> Dict[str, object]:
    return {"public_release": ACTIVE_PUBLIC_RELEASE, "bois_boris_main": ACTIVE_BOIS_BORIS_MAIN, "sima": ACTIVE_SIMA,
"boris_ego": ACTIVE_BORIS_EGO, "active_core_by_lang": ACTIVE_CORE_BY_LANG, "active_core_ru": ACTIVE_CORE_RU, "active_core_en":
ACTIVE_CORE_EN, "ego_default_core_mode": True, "ego_layer_is_core_part": True, "special_ego_mode_limits_superseded": True,
"mode10_trigger": MODE10_TRIGGER, "legacy_mode10_trigger": LEGACY_MODE10_TRIGGER, "mode10_special_mode_active": False,
"ego_expansion": EGO_EXPANSION, "ego_expansion_ru": EGO_EXPANSION_RU, "ego_acronym": EGO_EXPANSION, "ego_acronym_details":
ego_acronym_manifest(), "ego_default_core": ego_default_core_manifest(), "post_change_artifact_delivery":
post_change_artifact_manifest(), "response_format": response_format_manifest(), "anti_bureaucracy":
anti_bureaucracy_manifest(), "physiology_transit": physiology_transit_manifest(), "localized_term_understanding":
localized_term_understanding_manifest(), "goal_hierarchy": goal_hierarchy_manifest(), "goal_authority":
goal_authority_manifest(), "goal_class_authority": GOAL_CLASS_AUTHORITY, "goal_settable_by_boris_ego":
GOAL_SETTABLE_BY_BORIS_EGO, "final_goal_level": "G0", "derivative_goal_levels": ["G1", "G2", "G3", "G4"], "language_parity":
language_parity_manifest(), "basic_constraints": EGO_BASIC_ETHICAL_CONSTRAINTS, "core_calculation_categories":
sorted(CORE_CALCULATION_CATEGORIES), "physiology_allowed_categories": sorted(PHYSIOLOGY_ALLOWED_CATEGORIES), "immune_tests":
immune_test_manifest(), "physiology_integration": physiology_integration_manifest(), "changelog_hygiene":
changelog_hygiene_manifest(), "legal_code_registration": legal_code_registration_manifest(), "program_text_package":
program_text_package_manifest(), "program_text_integrity": program_text_integrity_manifest(), "archive_packaging":

```

```
archive_packaging_manifest(), "compact_archive_packaging": compact_archive_packaging_manifest(), "compact_packaging":
compact_packaging_manifest(), "archive_packaging_policy": archive_packaging_policy_manifest(), "compact_archive_policy":
compact_archive_policy_manifest())
```

```
def active_status_jsonable() -> Dict[str, object]:
    return active_status()
```

===== FILE: bois_sima_boris/locales.py =====

```
# SPDX-License-Identifier: MIT
LANGS = ['ar', 'zh', 'en', 'fr', 'ru', 'es']
CATALOG = {'en': {'docs_title': 'BOIS:SIMA&BORIS static documentation index', 'analysis_title': 'SIMA analysis',
'object_type': 'Object type', 'validation': 'Input validation passed', 'candidate': 'Candidate machine', 'confidence':
'Confidence', 'operome': 'Operome', 'risk_flags': 'Risk flags', 'no_risk': 'No high/medium structural flags detected by this
minimal analyzer.', 'limits': 'Limits', 'next_step': 'Next step'}, 'ru': {'docs_title': 'Индекс статической документации
BOIS:SIMA&BORIS', 'analysis_title': 'SIMA-анализ', 'object_type': 'Тип объекта', 'validation': 'Проверка входа пройдена',
'candidate': 'Кандидатная машина', 'confidence': 'Уверенность', 'operome': 'Опером', 'risk_flags': 'Флаги риска', 'no_risk':
'Высоких/средних структурных флагов этот минимальный анализатор не обнаружил.', 'limits': 'Ограничения', 'next_step':
'Следующий шаг'}, 'es': {'docs_title': 'Índice de documentación estática BOIS:SIMA&BORIS', 'analysis_title': 'Análisis SIMA',
'object_type': 'Tipo de objeto', 'validation': 'Validación de entrada superada', 'candidate': 'Máquina candidata',
'confidence': 'Confianza', 'operome': 'Operome', 'risk_flags': 'Señales de riesgo', 'no_risk': 'No se detectaron señales
estructurales altas/medias.', 'limits': 'Límites', 'next_step': 'Siguiendo paso'}, 'fr': {'docs_title': 'Index de
documentation statique BOIS:SIMA&BORIS', 'analysis_title': 'Analyse SIMA', 'object_type': 'Type d'objet', 'validation':
'Validation de l'entrée réussie', 'candidate': 'Machine candidate', 'confidence': 'Confiance', 'operome': 'Operome',
'risk_flags': 'Signaux de risque', 'no_risk': 'Aucun signal structurel élevé/moyen détecté.', 'limits': 'Limites',
'next_step': 'Prochaine étape'}, 'zh': {'docs_title': 'BOIS:SIMA&BORIS 静态文档索引', 'analysis_title': 'SIMA 分析',
'object_type': '对象类型', 'validation': '输入验证通过', 'candidate': '候选机器', 'confidence': '置信度', 'operome': 'Operome',
'risk_flags': '风险标志', 'no_risk': '此最小分析器未检测到高/中结构风险标志。', 'limits': '限制', 'next_step': '下一步'}, 'ar':
{'docs_title': 'فهرس الوثائق الثابتة BOIS:SIMA&BORIS', 'analysis_title': 'تحليل SIMA', 'object_type': 'نوع الكائن',
'validation': 'تم اجتياز التحقق من الإدخال', 'candidate': 'آلة مرشحة', 'confidence': 'الثقة', 'operome': 'Operome',
'risk_flags': 'إشارات المخاطر', 'no_risk': 'لم يكتشف هذا المحلل الأدنى إشارات بنوية عالية/متوسطة', 'limits':
'الحدود', 'next_step': 'الخطوة التالية'}}
```

```
def t(lang, key):
    lang = lang if lang in CATALOG else "en"
    return CATALOG[lang].get(key, CATALOG["en"].get(key, key))
```

===== FILE: bois_sima_boris/self_check.py =====

```
# SPDX-License-Identifier: MIT
from __future__ import annotations
from pathlib import Path
from typing import Dict, List

PROHIBITED_TOKENS = [
    "requests.", "urllib.request", "http.client", "socket.",
    "subprocess", "os.system", "eval(", "exec(", "pickle.loads",
    "paramiko", "ftplib", "smtplib"
]
SCAN_EXTENSIONS = {".py", ".yaml", ".yml"}
IGNORED_DIRS = {
    ".git", ".github", ".pytest_cache", "__pycache__", ".mypy_cache", ".ruff_cache",
    ".venv", "venv", "env", ".env", "build", "dist", ".eggs", "site-packages",
}
IGNORED_SUFFIXES = (".egg-info",)
```

```
def _ignored(path: Path) -> bool:
    parts = set(path.parts)
    if parts & IGNORED_DIRS:
        return True
    return any(part.endswith(IGNORED_SUFFIXES) for part in path.parts)
```

```
def run_self_check(root: Path) -> Dict[str, object]:
    findings: List[Dict[str, str]] = []
    root = root.resolve()
    for path in root.rglob("*"):
        if _ignored(path):
            continue
        if path.is_dir() or path.suffix.lower() not in SCAN_EXTENSIONS:
            continue
```

```
===== SOURCE LISTING PAGE 10 OF 52 =====
    rel = str(path.relative_to(root))
    # Do not flag this policy file for listing the prohibited tokens.
    if rel.endswith("self_check.py") or rel.endswith("SELF_CHECK_POLICY.md"):
        continue
    try:
        text = path.read_text(encoding="utf-8", errors="ignore")
```



```

except Exception as exc:
    findings.append({"file": rel, "token": "read_error", "detail": str(exc)})
    continue
for token in PROHIBITED_TOKENS:
    if token in text:
        findings.append({"file": rel, "token": token, "detail": "Potentially prohibited feature for public safe
baseline."})
    return {
        "ok": not findings,
        "policy": "Public safe baseline: no network, no shell, no dynamic code execution, no autonomous action.",
        "ignored_dirs": sorted(IGNORED_DIRS),
        "findings": findings,
    }

```

===== FILE: bois_sima_boris/sima.py =====

```

# SPDX-License-Identifier: MIT
"""Local SIMA analyzer.

```

This module deliberately avoids network calls, operating-system command execution, dynamic code execution, and autonomous action. It converts a local JSON description of an object/substrate into an auditable candidate philosophical-machine report.

```

"""
from __future__ import annotations

from dataclasses import dataclass, asdict
from typing import Any, Dict, List, Tuple

```

```

REQUIRED_TOP_LEVEL = ["object_id", "object_type", "domain", "value_interface", "substrate", "observations"]
SUBSTRATE_FIELDS = ["units", "channels", "memory", "costs", "failure_modes", "stop_authority"]
VALUE_FIELDS = ["goals", "constraints", "forbidden_losses"]

```

```

@dataclass
class Finding:
    id: str
    severity: str
    message: str
    recommendation: str

```

```

@dataclass
class Oper:
    id: str
    trigger: str
    distinction: str
    evidence: str
    carrier: str
    authority: str
    value_risk_gate: str

```

```

===== SOURCE LISTING PAGE 43 OF 52 =====
status = active_status()
assert status["public_release"] == "1.4.10"
assert status["boris_ego"] == "1.7"
assert status["ego_default_core_mode"] is True
assert status["special_ego_mode_limits_superseded"] is True
assert status["immune_tests"]["name"] == "BOIS EGO immune-test gate"
assert status["physiology_integration"]["system_physiology_allowed_to_be_large"] is True

```

```

def test_active_core_does_not_claim_actual_ai_consciousness():
    active_files = [ROOT / "core" / f"BORIS_EGO_LAYER_v1_7_{code}.md" for code in ["AR", "ZH", "EN", "FR", "RU", "ES"]]
    forbidden = ["AI is conscious", "program is conscious", "имеет субъективный опыт"]
    for path in active_files:
        text = path.read_text(encoding="utf-8")
        for phrase in forbidden:
            assert phrase not in text
        assert "not a factual claim" in text or "не является" in text or "不是" in text or "no afirma" in text or "n'affirme"
in text or "ليس" in text

```

===== FILE: tests/test_legal_code_registration_text_content_v147.py =====

```

# SPDX-License-Identifier: MIT
from pathlib import Path
import json
from bois_sima_boris.ego import active_status, route_calculation

```

```

ROOT = Path(__file__).resolve().parents[1]

```

```

def test_v148_legal_code_registration_artifacts_exist():
    assert (ROOT / "LEGAL_CODE_REGISTRATION_TEXT_CONTENT.md").exists()

```

```

legal = ROOT / "legal" / "copyright"
required = [
    "BOIS_PUBLIC_ARCHIVE_LEGAL_CODE_REGISTRATION_TEXT_CONTENT_v1_4_10_RU_EN.md",
    "BOIS_PUBLIC_ARCHIVE_SOURCE_CODE_FULL_v1_4_10.txt",
    "BOIS_PUBLIC_ARCHIVE_FULL_TEXT_CONTENT_v1_4_10.txt",
    "BOIS_PUBLIC_ARCHIVE_TEXT_MANIFEST_v1_4_10.json",
]
for name in required:
    assert (legal / name).exists(), name

def test_v148_legal_rule_is_active_and_core_routed():
    status = active_status()
    assert status["public_release"] == "1.4.10"
    assert status["boris_ego"] == "1.7"
    assert status["legal_code_registration"]["active"] is True
    assert status["legal_code_registration"]["attorney_selects_final_deposit_copy"] is True
    assert route_calculation("legal_code_registration_text_content") == "BOIS_CORE"

def test_v148_legal_manifest_identifies_source_and_text_content():
    manifest = json.loads((ROOT / "legal" / "copyright" /
"BOIS_PUBLIC_ARCHIVE_TEXT_MANIFEST_v1_4_10.json").read_text(encoding="utf-8"))
    assert manifest["public_release"] == "1.4.10"

===== SOURCE LISTING PAGE 44 OF 52 =====
    assert manifest["bois_boris_main"] == "2.23"
    assert manifest["boris_ego"] == "1.7"
    assert manifest["source_code_files_count"] >= 1
    assert any(item["path"] == "src/bois_sima_boris/ego.py" for item in manifest["source_code_files"])
    assert manifest["extraction_scope"] == "public archive textual content excluding generated self-referential full-text
file"

===== FILE: tests/test_localization.py =====

# SPDX-License-Identifier: MIT
from pathlib import Path
import re
import unittest
from bois_sima_boris.cli import main
from bois_sima_boris.ego import language_parity_manifest

ROOT = Path(__file__).resolve().parents[1]

FORBIDDEN_NON_ENGLISH_PHRASES = [
    "Minimal workflow:",
    "Describe an object in JSON.",
    "Validate it.",
    "Analyze it.",
    "## Workflow",
    "Define domain D and value interface V.",
    "Run validation and analysis.",
    "Review risk flags.",
    "Preserve rollback and stop conditions.",
    "## Documents",
    "## Equal language access",
    "No public access language is preferred.",
]

class TestLocalization(unittest.TestCase):
    def test_docs_languages(self):
        for lang in ["ar", "zh", "en", "fr", "ru", "es"]:
            self.assertEqual(main(["--lang", lang, "docs"]), 0)

    def test_un_language_user_docs_exist(self):
        required = language_parity_manifest()["required_user_docs"]
        for lang in language_parity_manifest()["languages"]:
            for name in required:
                self.assertTrue((ROOT / "docs" / lang / name).exists(), f"missing docs/{lang}/{name}")

    def test_active_core_mirrors_exist(self):
        for lang, rel in language_parity_manifest()["active_core_by_lang"].items():
            self.assertTrue((ROOT / rel).exists(), f"missing active core for {lang}: {rel}")

    def test_non_english_user_manuals_do_not_keep_old_english_workflow(self):
        for lang in ["ar", "zh", "fr", "ru", "es"]:
            for name in ["USER_MANUAL_SHORT.md", "USER_MANUAL_FULL.md", "INDEX.md"]:
                text = (ROOT / "docs" / lang / name).read_text(encoding="utf-8")
                for phrase in FORBIDDEN_NON_ENGLISH_PHRASES:
                    self.assertNotIn(phrase, text, f"{phrase!r} remains in docs/{lang}/{name}")

===== SOURCE LISTING PAGE 45 OF 52 =====

```

```

def test_start_here_links_exist(self):
    text = (ROOT / "START_HERE_UN.md").read_text(encoding="utf-8")
    refs = sorted(set(re.findall(r"`([^\`]+\md)`", text)))
    for rel in refs:
        self.assertTrue((ROOT / rel).exists(), f"broken START_HERE_UN link: {rel}")

if __name__ == "__main__":
    unittest.main()

===== FILE: tests/test_localized_term_understanding_v146.py =====

# SPDX-License-Identifier: MIT
import json
from pathlib import Path
from bois_sima_boris.ego import active_status, route_calculation, localized_term_understanding_manifest

ROOT = Path(__file__).resolve().parents[1]
LANGS = ["ar", "zh", "en", "fr", "ru", "es"]

def test_localized_term_understanding_rule_active_in_code_and_rule_map():
    data = json.loads((ROOT / "core" / "RULE_MAP_PUBLIC_v1_4_10.json").read_text(encoding="utf-8"))
    ids = [r if isinstance(r, str) else r["id"] for r in data["rules"]]
    assert "BR-108" in ids
    manifest = localized_term_understanding_manifest()
    assert manifest["active"] is True
    assert manifest["selected_non_english_response_language"] == "ru"
    assert route_calculation("localized_term_understanding") == "BOIS_CORE"

def test_active_core_and_language_docs_include_term_understanding():
    active = (ROOT / "core" / "ACTIVE_CORE_REFERENCE_v1_4_10.md").read_text(encoding="utf-8")
    assert "Localized Term Understanding Rule" in active
    for lang in LANGS:
        path = ROOT / "docs" / lang / "TERM_TRANSLATION_UNDERSTANDING.md"
        assert path.exists(), lang
        text = path.read_text(encoding="utf-8")
        if lang != "en":
            assert "[" in text and "]" in text

def test_ego_status_exposes_term_understanding():
    status = active_status()
    assert status["public_release"] == "1.4.10"
    assert status["boris_ego"] == "1.7"
    assert status["localized_term_understanding"]["active"] is True

def test_active_rule_requires_every_occurrence_not_first_only():
    from bois_sima_boris.ego import localized_term_understanding_manifest
    rule = localized_term_understanding_manifest()["rule"]
    assert "every occurrence" in rule

===== SOURCE LISTING PAGE 46 OF 52 =====
    assert "first appearance" not in rule
    active = (ROOT / "core" / "ACTIVE_CORE_REFERENCE_v1_4_10.md").read_text(encoding="utf-8")
    assert "при каждом появлении" in active

===== FILE: tests/test_program_text_package_v149.py =====

# SPDX-License-Identifier: MIT
from pathlib import Path
import json
from bois_sima_boris.ego import active_status, route_calculation
ROOT = Path(__file__).resolve().parents[1]

def test_v149_program_text_package_exists_and_is_active():
    status = active_status()
    assert status["public_release"] == "1.4.10"
    assert status["bois_boris_main"] == "2.23"
    assert status["boris_ego"] == "1.7"
    assert status["program_text_package"]["active"] is True
    assert status["program_text_package"]["sidecar_required"] is True
    assert route_calculation("program_text_package") == "BOIS_CORE"
    assert (ROOT / "PROGRAM_TEXT_PACKAGE.md").exists()

def test_v149_program_text_package_required_files_exist():
    base = ROOT / "legal" / "program_text"
    required = [
        "BOIS_PROGRAM_TEXT_PACKAGE_README_v1_4_10_RU_EN.md",
        "BOIS_PUBLIC_RELEASE_SOURCE_CODE_FULL_v1_4_10.txt",

```

```

        "BOIS_PUBLIC_RELEASE_SOURCE_CODE_DEPOSIT_FIRST10_LAST10_v1_4_10.txt",
        "BOIS_PUBLIC_RELEASE_SOURCE_CODE_DEPOSIT_FIRST10_LAST10_v1_4_10.md",
        "BOIS_PUBLIC_RELEASE_SOURCE_CODE_DEPOSIT_FIRST10_LAST10_v1_4_10.docx",
        "BOIS_PUBLIC_RELEASE_SOURCE_CODE_DEPOSIT_FIRST10_LAST10_v1_4_10.pdf",
        "BOIS_PROGRAM_TEXT_PACKAGE_MANIFEST_v1_4_10.json",
    ]
    for name in required:
        assert (base / name).exists(), name

def test_v149_program_text_manifest_covers_required_files():
    manifest = json.loads((ROOT / "legal" / "program_text" /
"BOIS_PROGRAM_TEXT_PACKAGE_MANIFEST_v1_4_10.json").read_text(encoding="utf-8"))
    assert manifest["public_release"] == "1.4.10"
    assert manifest["package_type"] == "PROGRAM_TEXT_PACKAGE"
    assert manifest["rule"] == "BR-117"
    names = {Path(item["path"]).name for item in manifest["files"]}
    assert "BOIS_PUBLIC_RELEASE_SOURCE_CODE_FULL_v1_4_10.txt" in names
    assert "BOIS_PUBLIC_RELEASE_SOURCE_CODE_DEPOSIT_FIRST10_LAST10_v1_4_10.docx" in names
    assert "BOIS_PUBLIC_RELEASE_SOURCE_CODE_DEPOSIT_FIRST10_LAST10_v1_4_10.pdf" in names

===== FILE: tests/test_redaction_public.py =====

# SPDX-License-Identifier: MIT
from pathlib import Path

def s(*codes):

===== SOURCE LISTING PAGE 47 OF 52 =====
    return "".join(chr(c) for c in codes)

FORBIDDEN = [
    s(65,108,97,115,107,97,32,83,116,111,110,101),
    s(65,108,97,115,107,97,32,83,116,111,110,101,115),
    s(65,108,97,115,107,97,95,83,116,111,110,101),
    s(65,108,97,115,107,97,95,83,116,111,110,101,115),
    s(69,103,111,114),
    s(84,101,109,110,105,107,111,118),
    s(1045,1075,1086,1088),
    s(1058,1077,1084,1085,1080,1082,1086,1074),
    s(101,103,111,114,46,116,101,109,110,105,107),
    s(103,109,97,105,108,46,99,111,109),
    s(83,104,111,114,101,108,105,110,101),
    s(74,101,119,105,115,104,32,65,117,116,111,110,111,109,111,117,115),
    s(1045,1040,1054),
]
SKIP_DIRS = {".venv", "venv", "__pycache__", ".pytest_cache", "build", "dist"}
TEXT_EXT = {".md", ".py", ".json", ".toml", ".txt", ".csv", ".cffi", ""}
ALLOWED_PRIVATE_PREFIXES = {"articles"}

def test_public_tree_has_no_known_private_tokens_outside_bundled_articles():
    root = Path(__file__).resolve().parents[1]
    hits = []
    for p in root.rglob("*"):
        rel_parts = p.relative_to(root).parts
        if any(part in SKIP_DIRS for part in p.parts):
            continue
        if rel_parts and rel_parts[0] in ALLOWED_PRIVATE_PREFIXES:
            # The article bundle intentionally contains public author metadata.
            continue
        if p.is_file() and p.suffix.lower() in TEXT_EXT:
            text = p.read_text(encoding="utf-8", errors="ignore")
            for token in FORBIDDEN:
                if token in text:
                    hits.append((str(p.relative_to(root)), "redacted-token"))
    assert hits == []

def test_bundled_article_exists_in_english_and_local_language():
    root = Path(__file__).resolve().parents[1]
    assert (root / "articles" / "en").exists()
    package_dirs = [p for p in (root / "articles").iterdir() if p.is_dir() and p.name.startswith("package_language_")]
    # Master packages may instead contain all language folders directly; language packages contain package_language_*.
    if package_dirs:
        assert any(d.glob("*.pdf") for d in package_dirs)
    else:
        assert any((root / "articles" / code).exists() for code in ("ar", "en", "es", "fr", "ru", "zh"))

===== FILE: tests/test_release_candidate_v141.py =====

# SPDX-License-Identifier: MIT
from pathlib import Path
import venv

===== SOURCE LISTING PAGE 48 OF 52 =====

```

```

import sys
from bois_sima_boris.ego import ACTIVE_PUBLIC_RELEASE, ACTIVE_BORIS_EGO, ACTIVE_BOIS_BORIS_MAIN, ACTIVE_CORE_BY_LANG

ROOT = Path(__file__).resolve().parents[1]
NON_EN = ["ar", "zh", "fr", "ru", "es"]
KNOWN_ENGLISH_BLOCKS = [
    "BORIS EGO is a minimal agent-like choice surface.",
    "This is not a factual claim",
    "Basic EGO ethical constraints",
    "The operator is the source of final goals.",
    "Harm to beings capable",
    "Lying and manipulation are forbidden.",
    "BORIS EGO-proposed messages are not user words.",
    "The overall system may have large physiology:",
    "Immune tests check:",
    "Standard mode:",
    "Mode 10 starts only",
    "Run: `python -m pytest`",
    "Test classes: philosophical",
    "Install Python 3.9+",
    "Unzip the package.",
]

def test_v142_active_versions_and_files():
    assert ACTIVE_PUBLIC_RELEASE == "1.4.10"
    assert ACTIVE_BOIS_BORIS_MAIN == "2.23"
    assert ACTIVE_BORIS_EGO == "1.7"
    assert (ROOT / "core" / "ACTIVE_CORE_REFERENCE_v1_4_10.md").exists()
    assert (ROOT / "core" / "RULE_MAP_PUBLIC_v1_4_10.json").exists()
    for rel in ACTIVE_CORE_BY_LANG.values():
        assert "v1_7" in rel
        assert (ROOT / rel).exists()

def test_active_non_english_ego_docs_have_no_known_english_blocks():
    for lang in NON_EN:
        paths = [ROOT / "docs" / lang / "BORIS_EGO_LAYER.md", ROOT / ACTIVE_CORE_BY_LANG[lang], ROOT / "docs" / lang /
"TESTING_AND_SELF_TESTING.md", ROOT / "docs" / lang / "INSTALL.md"]
        for path in paths:
            text = path.read_text(encoding="utf-8")
            for block in KNOWN_ENGLISH_BLOCKS:
                assert block not in text, f"{block!r} remains in {path.relative_to(ROOT)}"

def test_no_install_source_tree_shim_exists():
    assert (ROOT / "bois_sima_boris" / "__main__.py").exists()
    assert (ROOT / "bois_sima_boris" / "__init__.py").exists()

def test_dependency_free_build_backend_exists():
    backend = ROOT / "build_backend" / "bois_sima_boris_build_backend.py"
    assert backend.exists()
    text = backend.read_text(encoding="utf-8")
    assert "get_requires_for_build_editable" in text
    assert "return []" in text

===== FILE: tests/test_release_readiness_v139.py =====

===== SOURCE LISTING PAGE 49 OF 52 =====

# SPDX-License-Identifier: MIT
from pathlib import Path
import re

from bois_sima_boris.ego import (
    ACTIVE_PUBLIC_RELEASE, ACTIVE_BORIS_EGO, ACTIVE_BOIS_BORIS_MAIN,
    ACTIVE_CORE_BY_LANG, language_parity_manifest, route_calculation,
)
from bois_sima_boris.cli import main

ROOT = Path(__file__).resolve().parents[1]
LANGS = ["ar", "zh", "en", "fr", "ru", "es"]

def test_v139_active_versions_and_core_files():
    assert ACTIVE_PUBLIC_RELEASE == "1.4.10"
    assert ACTIVE_BOIS_BORIS_MAIN == "2.23"
    assert ACTIVE_BORIS_EGO == "1.7"
    for lang, rel in ACTIVE_CORE_BY_LANG.items():
        assert (ROOT / rel).exists(), f"missing active core for {lang}: {rel}"

def test_user_instruction_alias_parity():
    required = set(language_parity_manifest()["required_user_docs"])
    required.update({"USER_INSTRUCTIONS_SHORT.md", "USER_INSTRUCTIONS_FULL.md"})
    for lang in LANGS:
        for name in required:
            assert (ROOT / "docs" / lang / name).exists(), f"missing docs/{lang}/{name}"

```

```

def test_cli_docs_lists_instruction_aliases_and_existing_paths(capsys):
    for lang in LANGS:
        assert main(["docs", "--lang", lang]) == 0
        out = capsys.readouterr().out
        assert f"docs/{lang}/USER_INSTRUCTIONS_SHORT.md" in out
        assert f"docs/{lang}/USER_INSTRUCTIONS_FULL.md" in out
        for line in out.splitlines():
            if line.startswith("- "):
                rel = line[2:].strip()
                assert (ROOT / rel).exists(), f"CLI listed missing path for {lang}: {rel}"

def test_historical_ego_links_resolve():
    for rel in [
        "core/BORIS_EGO_LAYER_v0_1_RU.md", "core/BORIS_EGO_LAYER_v0_1_EN.md",
        "core/BORIS_EGO_LAYER_v0_2_RU.md", "core/BORIS_EGO_LAYER_v0_2_EN.md",
        "core/BORIS_EGO_LAYER_v0_3_RU.md", "core/BORIS_EGO_LAYER_v0_3_EN.md",
    ]:
        assert (ROOT / rel).exists(), rel

def test_release_readiness_routes_to_core():
    assert route_calculation("release_readiness_gate") == "BOIS_CORE"

===== SOURCE LISTING PAGE 50 OF 52 =====

===== FILE: tests/test_rule_conflict_v144.py =====

# SPDX-License-Identifier: MIT
import json
from pathlib import Path
from bois_sima_boris.ego import active_status, can_boris_ego_edit, can_boris_ego_set_goal

ROOT = Path(__file__).resolve().parents[1]

def test_rule_map_has_unique_new_rules_and_active_defaults():
    data = json.loads((ROOT / "core" / "RULE_MAP_PUBLIC_v1_4_10.json").read_text(encoding="utf-8"))
    ids = [r if isinstance(r, str) else r["id"] for r in data["rules"]]
    assert len(ids) == len(set(ids))
    for rid in ["BR-102", "BR-103", "BR-104", "BR-105", "BR-106", "BR-107", "BR-108"]:
        assert rid in ids
    assert data["default_response_format"] == "MAXIMUM_OPERATIONAL_RESPONSE"
    assert data["physiology_as_core_transit"] is True

def test_ego_basic_ethics_and_goal_hierarchy_still_boundaries():
    assert can_boris_ego_edit("L0", explicit_operator_authority=True)["allowed"] is False
    assert can_boris_ego_edit("L1", explicit_operator_authority=True, core_change_protocol=False)["allowed"] is False
    assert can_boris_ego_edit("L1", explicit_operator_authority=True, core_change_protocol=True)["allowed"] is True
    assert can_boris_ego_set_goal("G0")["allowed"] is False
    assert can_boris_ego_set_goal("G1", traceable_to_operator_final_goal=True)["allowed"] is True
    assert can_boris_ego_set_goal("G2", traceable_to_operator_final_goal=False)["allowed"] is False

def test_mode10_not_active_and_not_contradicting_default_ego():
    status = active_status()
    assert status["ego_default_core_mode"] is True
    assert status["mode10_special_mode_active"] is False
    assert status["ego_default_core"]["mode10_counter_required"] is False
    assert status["ego_default_core"]["mode10_extension_prompt_required"] is False

===== FILE: tests/test_self_check_hygiene.py =====

# SPDX-License-Identifier: MIT
from pathlib import Path
from bois_sima_boris.self_check import run_self_check

def test_self_check_ignores_local_virtualenv(tmp_path: Path):
    root = tmp_path / "pkg"
    root.mkdir()
    (root / "safe.py").write_text("print('ok')\n", encoding="utf-8")
    venv = root / ".venv" / "lib" / "python" / "site-packages" / "example"
    venv.mkdir(parents=True)
    prohibited_import = "import sub" + "process\n"
    (venv / "bad.py").write_text(prohibited_import, encoding="utf-8")
    result = run_self_check(root)

```

===== SOURCE LISTING PAGE 51 OF 52 =====

```
assert result["ok"] is True
assert result["findings"] == []
```

===== FILE: tests/test_sima.py =====

```
# SPDX-License-Identifier: MIT
import json
import unittest
from pathlib import Path
from bois_sima_boris.sima import analyze_object, validate_object

class TestSIMA(unittest.TestCase):
    def test_example_ai_workflow_passes(self):
        obj = json.loads(Path("examples/example_local_workflow.json").read_text(encoding="utf-8"))
        ok, findings = validate_object(obj)
        self.assertTrue(ok, [f.__dict__ for f in findings])
        result = analyze_object(obj)
        self.assertGreaterEqual(result["operome"]["count"], 3)
        self.assertTrue(result["candidate_machines"][0]["M"]["C_closure_rule"])

    def test_incomplete_object_fails(self):
        ok, findings = validate_object({"object_id": "bad"})
        self.assertFalse(ok)
        self.assertTrue(any(f.severity == "high" for f in findings))

if __name__ == "__main__":
    unittest.main()
```

===== FILE: tests/test_un_language_parity_strict_v05.py =====

```
# SPDX-License-Identifier: MIT
from pathlib import Path
import re
from bois_sima_boris.cli import main
from bois_sima_boris.ego import language_parity_manifest, route_calculation

ROOT = Path(__file__).resolve().parents[1]
NON_EN = ["ar", "zh", "fr", "ru", "es"]
FORBIDDEN_WORKFLOW_FRAGMENTS = [
    "Minimal workflow",
    "Describe an object",
    "Validate it",
    "Analyze it",
    "Define domain",
    "Run validation",
    "Review risk flags",
    "Preserve rollback",
    "Do not convert the output",
]
```

===== SOURCE LISTING PAGE 52 OF 52 =====

```
def _strip_code_blocks(text: str) -> str:
    return re.sub(r"```.*?```", "", text, flags=re.S)

def test_required_language_docs_and_neutrality_doc_exist():
    required = set(language_parity_manifest()["required_user_docs"])
    required.add("LANGUAGE_NEUTRALITY.md")
    for lang in language_parity_manifest()["languages"]:
        for name in required:
            assert (ROOT / "docs" / lang / name).exists(), f"missing docs/{lang}/{name}"

def test_cli_docs_lists_only_existing_files(capsys):
    for lang in language_parity_manifest()["languages"]:
        assert main(["docs", "--lang", lang]) == 0
        out = capsys.readouterr().out
        for line in out.splitlines():
            if not line.startswith("- "):
                continue
            rel = line[2:].strip()
            assert (ROOT / rel).exists(), f"CLI listed missing file for {lang}: {rel}"

def test_non_english_user_manuals_do_not_retain_core_english_workflow_fragments():
    for lang in NON_EN:
```

```
for name in ["USER_MANUAL_SHORT.md", "USER_MANUAL_FULL.md"]:
    text = _strip_code_blocks((ROOT / "docs" / lang / name).read_text(encoding="utf-8"))
    for frag in FORBIDDEN_WORKFLOW_FRAGMENTS:
        assert frag not in text, f"{frag!r} remains in docs/{lang}/{name}"
```

```
def test_non_english_indexes_localize_main_section_labels():
    for lang in NON_EN:
        text = (ROOT / "docs" / lang / "INDEX.md").read_text(encoding="utf-8")
        assert "## Documents" not in text
        assert "## Equal language access" not in text
        assert "No public access language is preferred" not in text
```

```
def test_language_parity_conflicts_and_instruction_completeness_route_to_core():
    assert route_calculation("language_parity_conflict") == "BOIS_CORE"
    assert route_calculation("localized_instruction_completeness") == "BOIS_CORE"
```